Proprietà degli algoritmi di sorting, Ricorsione

Materia	Algoritmi
Anno	Secondo Anno
Data	@October 4, 2024
Studiato	Done

Consideriamo gli algoritmi di sorting:

- selection sort;
- bubble sort;
- insertion sort;

Nel selection sort, ricerchiamo ad ogni iterazione il massimo dell'array. Dopo la prima iterazione, il massimo viene collocato nella posizione corretta e controlliamo gli $n-1$ elementi rimasti. Chiaramente, la complessità di questo algoritmo è sempre $O(n^2)$, perché anche supponendo che l'array sia ordinato, dobbiamo sempre controllare prima n elementi, poi $n-1$ e così via. Lo pseudocodice è il seguente:

```
selectionSort(A, n):
    for i = 1 to n do
        min = i
        for j = i to n do
            if A[j] < A[min] then
                min = j
        temp = A[i]
        A[i] = min
        min = temp
```

Nel bubble sort, l'ordinamento avviene tramite "bolle", ossia si considerano due elementi contigui e si scambiano di posto quando il primo è più grande del successivo. **Senza considerare la guardia**, abbiamo sempre $O(n^2)$, perché abbiamo sempre due `for` annidati. Lo pseudocodice è:

```
bubbleSort(A, n):
   for i = 1 to n do:
      for j = 1 to n do:
         if a[j] > a[j+1] then
            temp = a[j]
            a[j] = a[j+1]
            a[j+1] = temp
```

Infine, nell'insertion sort eseguiamo scambi in avanti fino a quando non troviamo la posizione corretta dell'elemento considerato. Nel caso migliore l'insertion sort ha tempo lineare, perché il secondo ciclo, se l'array è ordinato, ha tempo costante.

L'insertion sort è l'unico algoritmo che riesce a sfruttare alcune proprietà dell'input. In particolare, la complessità dell'insertion sort nel caso medio è considerabile come $O(nk)$, dove k è una **costante corrispondente alla distanza massima tra un elemento e la sua posizione finale**.

La capacità precedentemente descritta prende il nome di **adattività.**

> *Un algoritmo è **adattivo** se sfrutta alcune proprietà dell'input.*

Invece, tutti e tre gli algoritmi operano **in loco**.

> *Un algoritmo opera **in loco** se non necessita di memoria aggiuntiva.*

Un'altra proprietà importante è la **stabilità**.

> *Un algoritmo di ordinamento si dice **stabile** se mantiene l'ordinamento relativo degli elementi che hanno la stessa chiave.*

Ad esempio, se in un array sono presenti elementi duplicati, **abbiamo più soluzioni**, perché gli elementi che si ripetono tra di loro possono essere disposti in maniera diversa (sebbene, alla fine, l'array finale è sempre lo stesso). Se aggiungiamo un'etichetta agli elementi duplicati, allora la soluzione corretta è quella che preserva l'ordinamento delle etichette:

$$\begin{gathered}\left[1_A, 2_A, 1_B, 3_A, 5_A, 7_A\right] \\ \downarrow \\ \left[1_A, 1_B, 2_A, 3_A, 5_A, 7_A\right]\end{gathered}$$

Nell'esempio sopra, la soluzione corretta è $1_A, 1_B$, e non viceversa. Se un algoritmo di ordinamento gode di questa proprietà, allora è stabile.
Il selection sort potrebbe essere implementato in maniera stabile modificando la funzione di massimo:

```
max(A,n):
   m = 0
   for i = 1 to n-1 do
      if A[m] <= A[i] then
         m = i
   return m
```

Tuttavia, **solo questa modifica non basta**, perché nel momento in cui facciamo uno scambio vi è il rischio di modificare "l'ordinamento" degli elementi con la stessa chiave. Dovremmo modificare quindi l'algoritmo di scambio, scambiando gli elementi $A[i]$ e $A[i+1]$ fino a quando non arriviamo alla posizione corretta.

Proprio per quanto detto precedentemente, bubble sort e insertion sort, che operano con scambi tra elementi adiacenti, sono **stabili**.

Ordinamenti ricorsivi

Analizziamo ora il quick sort e il merge sort. Nel caso migliore, entrambi hanno complessità $O(n \log n)$, mentre nel caso peggiore, il quick sort ha complessità $O(n^2)$, mentre il merge sort ha complessità $O(n \log n)$. Il quick sort, inoltre, ha il caso peggiore **quando l'array è ordinato**, mentre si comporta bene quando **l'array**

ha struttura ad alta entropia (ossia è random). Entrambi gli algoritmi quindi **non sono adattivi**.

Il quick sort è **in loco**, sebbene esegua chiamate ricorsive e pertanto occupa memoria **non esplicita**. Il merge sort, invece, non è in loco.

Il quick sort, per lo stesso motivo del selection sort, **non può essere implementato in maniera stabile**, mentre il merge sort, nel momento in cui confronto due elementi, posso implementare un confronto di etichette, e quindi dare precedenza all'elemento con etichetta più bassa, quindi è implementabile in maniera stabile.

Ricorsione

La ricorsione è una tecnica di programmazione in cui **una procedura chiama se stessa per risolvere un problema**. Tale problema viene risolto scomponendolo in problemi più piccoli dello stesso tipo.

Sia P un problema scomponibile in sotto-problemi P_1, P_2. Supponiamo di **avere le soluzioni dei problemi** P_1, P_2. Allora **uniamo le due soluzioni in una soluzione più grande** S**, soluzione del problema** P.

Trovare questa soluzione, in realtà, **non sempre è semplice**, ma il vantaggio della ricorsione è il fatto che **possiamo non sapere la soluzione diretta di** P **e avere semplicemente la soluzione di un caso base** B**,** ossia un **caso in cui la soluzione sia diretta e semplice**.

Nel caso del merge sort, l'ordinamento funziona in questo modo. Dividiamo l'array in due parti uguali, e continuiamo fino a quando non abbiamo 1 solo elemento dell'array (**caso base**). **Supponendo che due array più piccoli siano già ordinati**, posso unirli con la procedura di `merge()`.

Generalmente nella ricorsione eseguiamo due passi fondamentali:

- **suddivisione del problema**;
- **unione delle soluzioni**.

Consideriamo il quick sort. Scelto un pivot, suddividiamo l'array in due parti:

- la partizione sinistra contiene gli elementi **più piccoli del pivot**;

- la partizione destra contiene gli elementi **più grandi del pivot**.

Il lavoro maggiore viene fatto nel partizionamento, che ha complessità $O(n)$. Procedendo ricorsivamente, l'unione è **costante** perché **non vi è una reale operazione di merge visto che si opera in loco**.

A parità di algoritmo, implementare un algoritmo in maniera ricorsiva porta a tempi di esecuzione superiori rispetto alla iterativa. Tuttavia permette soluzioni più semplici.

Selection sort ricorsivo

Il nostro caso base è semplicemente se l'array ha lunghezza 1 o 0. Una volta che troviamo il massimo, richiamiamo la funzione per gli elementi rimanenti. Il codice sarà il seguente:

```
selectionSort(A, n)
    if n <= 1 then return
    m = max(A, n)
    swap(A, m, n-1)
    selectionSort(A, n-1)
```

Insertion sort ricorsivo

Il nostro caso base, come prima, è sempre lunghezza 0 o 1 dell'array. Richiamiamo la funzione prima di eseguire l'operazione di inserimento nel seguente modo:

```
insertionSort(A, n)
    if n <= 1 then return
    insertionSort(a, n-1)
    insert(A, n-1)
```